



**Universidad
Europea**

UNIVERSIDAD EUROPEA DE ANDALUCÍA

ESCUELA POLITÉCNICA

**MÁSTER UNIVERSITARIO EN ANÁLISIS DE GRANDES
VOLÚMENES DE DATOS (BIG DATA)**

TRABAJO FIN DE MÁSTER

**DESARROLLO DE UN SISTEMA
ESCALABLE DE MICROSERVICIOS PARA EL
PROCESAMIENTO Y ANÁLISIS DE DATOS
IOT**

BORIS RENEE PATERNINA PEREZ

Dirigido por

VICTOR GOMEZ GUIRADO

CURSO 2025 - 2026

TÍTULO: DESARROLLO DE UN SISTEMA ESCALABLE DE MICROSERVICIOS PARA EL
PROCESAMIENTO Y ANÁLISIS DE DATOS IOT

AUTOR: BORIS RENEE PATERNINA PEREZ

TITULACIÓN: MÁSTER UNIVERSITARIO EN ANÁLISIS DE GRANDES VOLÚMENES DE
DATOS (BIG DATA)

DIRECTOR DEL PROYECTO: VICTOR GOMEZ GUIRADO

FECHA: Septiembre de 2026

RESUMEN

El resumen tiene entre 150-250 palabras. Resumir consiste en ofrecer información sobre cómo, dónde, cuándo y por qué se aplica el proyecto. Se realiza al finalizar el trabajo.

Debe incluir:

- Resumen del problema planteado y las aportaciones más importantes del proyecto al respecto
- Indica si procede si el proyecto se realizó en colaboración con una empresa, indicando nombre (siempre que haya autorización expresa por la empresa), y el sector industrial o empresarial
- Principales resultados del proyecto (diseño de un sistema para. . . , desarrollo e implementación de una solución para. . . , análisis de. . . , modelo de. . .)
- Resume las conclusiones más importantes del trabajo realizado
- El resumen NO
- Da una información genérica
- Se refiere a datos aportados en el texto del proyecto.

Palabras clave: hasta un máximo de 6 conceptos (un concepto puede conllevar una o más palabras). Incluye tecnologías que hayas utilizado, conceptos relevantes del ámbito científico-técnico, y conceptos de la industria (algunos ejemplos: machine learning, Big Data, Smart City, visión artificial, social media, Twitter, computación afectiva, transformación digital, GDPR, DevOps, EEG, . . .)

ABSTRACT

Resumen en inglés.

Keywords: Palabras clave en inglés

Dedicado a mis padres

Índice general

1. RESUMEN DEL PROYECTO	9
1.1. Contexto y justificación	9
1.2. Planteamiento del problema	9
1.3. Objetivos del proyecto	9
1.4. Resultados obtenidos	9
1.5. Estructura de la memoria	9
2. ANTECEDENTES Y ESTADO DEL ARTE	10
2.1. Antecedentes	10
2.1.1. El crecimiento de la analítica de datos en el Internet de las Cosas	10
2.1.2. Categorización de la analítica de datos IoT	10
2.1.3. Arquitecturas de referencia: Lambda y Kappa	11
2.2. Estado del arte	13
2.2.1. Ecosistema de herramientas de código abierto en el ámbito de los datos IoT	13
2.2.2. Retos abiertos: seguridad, interoperabilidad y especialización técnica . .	15
2.2.3. Criterios de selección de herramientas de procesamiento streaming . . .	15
2.3. Contexto y justificación	16
2.4. Planteamiento del problema	17
3. OBJETIVOS	19
3.1. Alcance del proyecto	19
3.2. Tareas a desarrollar	19
3.3. Objetivos	19
3.3.1. Objetivo 1: Garantizar la ingesta fiable de telemetría en tiempo real . . .	20
3.3.2. Objetivo 2: Garantizar la gobernanza y evolución controlada del esque- ma de datos	20
3.3.3. Objetivo 3: Procesar y enriquecer los datos en streaming con baja latencia	20
3.3.4. Objetivo 4: Ofrecer visualización diferenciada operacional y analítica . .	20
3.3.5. Objetivo 5: Validar la resiliencia del sistema	21
3.4. Beneficios del proyecto	21
4. ÉTICA Y COMPROMISO SOCIAL	22
4.1. Igualdad de género	22
4.2. Objetivos de Desarrollo Sostenible	22
5. DESARROLLO DEL PROYECTO	24
5.1. Planificación del proyecto	24
5.2. Descripción de la solución, metodologías y herramientas empleadas	24
5.2.1. Visión general de la arquitectura	24

5.2.2. Fuente de datos y variables utilizadas	26
5.2.3. Criterios de selección de herramientas	27
5.2.4. Estructura del repositorio	28
5.2.5. Preparación de los datos	29
5.2.6. Simulador de telemetría	29
5.2.7. Broker MQTT (Eclipse Mosquitto)	30
5.2.8. Bridge MQTT → Kafka	31
5.2.9. Registro de esquemas (Apicurio Registry)	32
5.2.10. Apache Kafka	34
5.2.11. Apache Spark Structured Streaming	34
5.3. Recursos requeridos	35
5.4. Presupuesto	35
5.5. Viabilidad	36
5.6. Resultados del proyecto	36
6. DISCUSIÓN	37
7. CONCLUSIONES	38
7.1. Conclusiones del trabajo	38
7.2. Conclusiones personales	38
8. FUTURAS LÍNEAS DE TRABAJO	39
Bibliografía	40
9. ANEXOS	42

Índice de Figuras

2.1. Categorías, enfoques y componentes de la analítica de datos IoT	11
2.2. Arquitectura Lambda	12
2.3. Arquitectura Kappa	12
5.1. Arquitectura del pipeline IoT	25
5.2. Esta es una imagen de ejemplo.	36

Índice de Tablas

2.1. Principales brokers MQTT con licencia aprobada por la Open Source Initiative .	13
2.2. Principales plataformas de mensajería persistente con licencia aprobada por la Open Source Initiative	14
5.1. Cronograma y esfuerzo de las actividades del proyecto	24
5.2. Variables de la tabla de hechos (ashrae_telemetry.parquet) empleadas. . .	26
5.3. Variables de la tabla de dimensión (ashrae_buildings.parquet) empleadas. .	27
5.4. Variables de la línea base por sensor (ashrae_sensor_baseline.parquet) empleadas.	27
5.5. Relación entre el factor de aceleración y el ritmo de publicación.	29
5.6. Campos del mensaje JSON publicado por el simulador.	30
5.7. Costes del proyecto	36

Capítulo 1. RESUMEN DEL PROYECTO

Este capítulo es un resumen, y no debe incluir el detalle. Existen capítulos específicos para añadir detalle de cada apartado. por tanto, únicamente se hace un resumen del trabajo completo, siguiendo la estructura marcada por las secciones de este capítulo.

Ocupa un máximo de 1 página.

1.1 Contexto y justificación

Resumen del por qué del proyecto y del contexto en el que se desarrolla. Está relacionado con el capítulo 2.

1.2 Planteamiento del problema

Resumen del planteamiento del problema que pretendemos solucionar con nuestro proyecto. Se busca una pregunta motriz a la que el trabajo pretende dar respuesta. Aquí se puede hablar del estado del arte o antecedentes de dicho problema.

Indica si el proyecto se ha desarrollado para resolver un problema específico empresarial, o para investigar en una cuestión específica del ámbito científico-técnico, y/o incluye de innovación.

1.3 Objetivos del proyecto

Incluye aquí solo un resumen de objetivos. En el capítulo correspondiente a objetivos se detallan tanto el general como los específicos.

1.4 Resultados obtenidos

Incluye el resumen de los resultados obtenidos al final de tu proyecto. En el capítulo correspondiente a resultados se detallan estos resultados.

1.5 Estructura de la memoria

Describe muy brevemente los capítulos y su contenido, con tus propias palabras.

Capítulo 2. ANTECEDENTES Y ESTADO DEL ARTE

2.1 Antecedentes

2.1.1. El crecimiento de la analítica de datos en el Internet de las Cosas

El despliegue masivo de dispositivos conectados ha convertido a la analítica de datos en el elemento central para transformar la telemetría generada por sensores e infraestructuras inteligentes en información accionable. Como señala Y. Liu [1], el crecimiento exponencial del volumen, la velocidad y la variedad de los datos generados por miles de millones de dispositivos ha superado la capacidad de los enfoques tradicionales de procesamiento orientado a lotes, obligando a la creación y adopción de arquitecturas capaces de ingerir, procesar y almacenar flujos continuos de datos en tiempo real.

Junto a los factores tecnológicos de la transición impulsada por el crecimiento exponencial de los datos IoT, debe considerarse que el valor de dichos datos depende de la latencia con la que estos se procesan. Por ejemplo, los datos de temperatura de un motor industrial pierden gran parte de su utilidad inicial si se analizan horas después de haber sido generados, cuando la ventana para prevenir el evento ya se ha cerrado. Este factor temporal motiva la centralidad que ha adquirido el procesamiento por flujos (*stream processing*) versus el procesamiento por lotes clásico para los pipelines IoT modernos.

El proceso de análisis de datos IoT abarca la recolección, el procesamiento, el almacenamiento y la extracción de conocimiento, y su relevancia se ha extendido a dominios tan diversos como la salud, las ciudades inteligentes, la automatización industrial, el monitoreo ambiental y el transporte inteligente (Y. Liu [1]).

2.1.2. Categorización de la analítica de datos IoT

Y. Liu [1] propone un marco de clasificación que resulta útil para situar cualquier pipeline dentro del panorama general de la analítica de datos IoT. El marco de clasificación distingue tres dimensiones globales, tal como se muestra en la Figura 2.1:

- **Categorías de análisis**, según el momento y el propósito del procesamiento: análisis descriptivo (qué ha ocurrido), análisis en tiempo real (qué está ocurriendo), análisis predictivo (qué es probable que ocurra) y análisis prescriptivo (qué debería hacerse al respecto).
- **Enfoques de procesamiento de datos**: Computación en la nube (*cloud-based*), computación en el borde (*edge*) y computación en la niebla (*fog computing*), cada uno con distintos compromisos entre latencia, ancho de banda y capacidad de cómputo disponible.

- **Componentes funcionales:** recolección e ingesta de datos, almacenamiento y gestión, preprocesamiento y transformación, y finalmente analítica y aprendizaje automático.

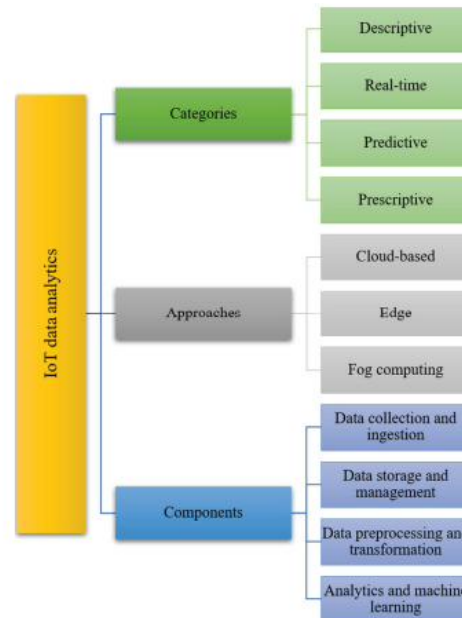


Figure 1 Overview of IoT data analytics categories, approaches, and components.

Figura 2.1. Categorías, enfoques y componentes de la analítica de datos IoT. Reproducida de Y. Liu [1], distribuida bajo licencia Creative Commons (CC BY).

2.1.3. Arquitecturas de referencia: Lambda y Kappa

En la literatura sobre procesamiento de grandes volúmenes de datos en streaming destacan dos patrones arquitectónicos ampliamente documentados. La arquitectura Lambda fue propuesta originalmente por N. Marz [2] en 2011 (posteriormente detallada junto con J. Warren [3]), que parte de la premisa de que toda consulta puede expresarse como una función sobre la totalidad de los datos ($query = function(all\ data)$), pero calcular esa función *en tiempo real* sobre el dataset completo resulta prohibitivamente costoso. Para resolver esto, se implementa una capa de procesamiento por lotes (que recalcula vistas precisas sobre el histórico completo), junto a una capa de velocidad (que ofrece vistas aproximadas de baja latencia sobre los datos más recientes), que después se fusionan en una capa de servicio.

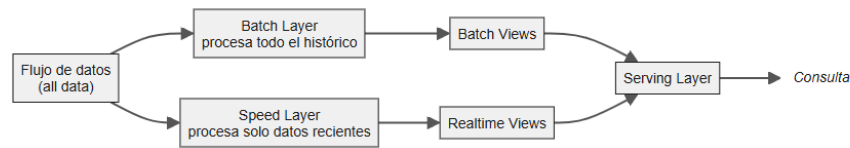


Figura 2.2. Arquitectura Lambda: las capas batch y speed se fusionan en la capa de servicio. Elaboración propia a partir de [2], [3].

No obstante la amplia adopción de la arquitectura Lambda, J. Kreps [4] señala que esta exige implementar y mantener la misma lógica de transformación por duplicado: una en el sistema batch y otra en el de streaming; y sincronizar ambas transformaciones resulta muy costoso operativamente. H. Cao y M. Wachowicz [5] añaden que la dependencia de infraestructura de cómputo intensivo hace esta arquitectura poco adecuada para muchos escenarios IoT.

Como alternativa, J. Kreps [4] propuso en 2014 la arquitectura Kappa, que elimina la capa por lotes y utiliza un flujo reproducible (*replayable stream*) como fuente de verdad única. Cuando hace falta reprocesar el flujo (por un cambio de lógica o corrección de un bug), se lanza una nueva instancia de job de streaming (con una versión mejorada del código, corriendo sobre el mismo framework) que se ejecuta reproduciendo el flujo desde el principio, escribiendo su resultado en una nueva tabla de salida y en paralelo a la tabla que ya está en producción; una vez que el nuevo job se sincroniza al presente, la aplicación se redirige a leer de la tabla nueva, y solo entonces se puede retirar la tabla y el job antiguos [4].

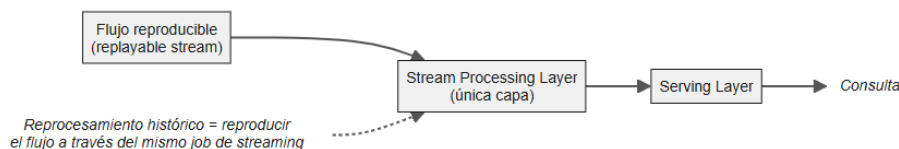


Figura 2.3. Arquitectura Kappa: una única capa de streaming, con reprocesamiento histórico mediante reproducción del flujo. Elaboración propia a partir de [4].

Ejemplos recientes de implementación práctica de Kappa en contextos IoT incluyen a S. Shinde [6], quien construye un pipeline de monitorización de temperatura y detección de anomalías con Kafka, Spark Streaming y Streamlit, y a G. Zefkilis [7], que documenta una variante con Kafka, Spark Streaming, StarRocks y MinIO sobre Docker. Cabe señalar, no obstante, que ambas citas son tutoriales técnicos de un único autor.

2.2 Estado del arte

2.2.1. Ecosistema de herramientas de código abierto en el ámbito de los datos IoT

El ecosistema de herramientas de código abierto disponibles para cada capa del pipeline ha madurado considerablemente, como ilustran las implementaciones de S. Shinde [6] y G. Zefkilis [7] comentadas en la sección anterior.

En la capa de ingesta tenemos varios estándares disponibles para los protocolos de comunicación con los dispositivos IoT; entre ellos **MQTT** (ISO/IEC 20922) ofrece un modelo de publicación-suscripción por tópicos con broker central y niveles de calidad de servicio configurables, adecuado para dispositivos numerosos y de conectividad intermitente; **CoAP** (RFC 7252) sigue un modelo de petición-respuesta sobre UDP, pensado para dispositivos de recursos muy limitados, y solo incorpora publicación-suscripción mediante extensiones; **AMQP 1.0** (ISO/IEC 19464) está orientado a mensajería empresarial con mayores garantías y coste de implementación; y **DDS** (especificación OMG) adopta un modelo descentralizado sin broker, propio de sistemas de control en tiempo real.

Estableciendo MQTT como protocolo para este trabajo, los brokers disponibles para la recolección de telemetría se resumen en la Tabla 2.1.

Broker	Licencia	Lenguaje	Observaciones
Eclipse Mosquitto	EPL 2.0 / EDL 1.0	C	Implementación de referencia de la Eclipse Foundation; exclusivamente MQTT, de huella mínima
HiveMQ Community Ed.	Apache 2.0	Java	La extensión de integración con Kafka es producto comercial
VerneMQ	Apache 2.0	Erlang	Orientado a despliegues distribuidos, con clustering nativo
NanoMQ	MIT	C	Diseñado para el borde de la red
Apache ActiveMQ Artemis	Apache 2.0	Java	Multiprotocolo: AMQP 1.0, MQTT 3.1.1 y 5.0, y STOMP
RabbitMQ	MPL 2.0	Erlang	Broker de colas con soporte de MQTT 3.1, 3.1.1 y 5.0

Tabla 2.1. Principales brokers MQTT con licencia aprobada por la Open Source Initiative

De este inventario conviene excluir EMQX [8], habitualmente citado entre las opciones de licencia libre: desde su versión 5.9.0 el proyecto abandonó la licencia Apache 2.0 en favor de una licencia *source-available* no aprobada por la Open Source Initiative.

Para el acoplamiento entre los brokers de recolección y las plataformas de mensajería persistente, no existe una solución de código abierto integrada en el propio broker: los que ofrecen un puente nativo lo hacen desde una edición comercial [9] o bajo licencias *source-available* [8]. En consecuencia, toda arquitectura restringida a herramientas de código abierto debe incorporar un componente adicional de *bridging*. Este puede ser un conector genérico, como el conector MQTT de Stream Reactor para Kafka Connect [10], o un conector desarrollado a la medida si se quiere asignar funcionalidades adicionales a dicho componente.

Para la capa de almacenamiento reproducible del flujo que alimentará a la capa de procesamiento, la oferta de plataformas de mensajería persistente es igualmente amplia, como recoge la Tabla 2.2.

Plataforma	Licencia	Lenguaje	Observaciones
Apache Kafka	Apache 2.0	Java/Scala	Registro particionado con retención configurable y desplazamiento por consumidor
Apache Pulsar	Apache 2.0	Java	Separa cómputo y almacenamiento mediante BookKeeper; multi-tenencia y geo-replicación nativas
Apache RocketMQ	Apache 2.0	Java	Reproducción de mensajes por tiempo o por desplazamiento; ecosistema concentrado en Asia
NATS (JetStream)	Apache 2.0	Go	Binario único sin dependencia de JVM; ecosistema de conectores más reducido
RabbitMQ Streams	MPL 2.0	Erlang	Registro persistente añadido sobre un broker de colas

Tabla 2.2. Principales plataformas de mensajería persistente con licencia aprobada por la Open Source Initiative

Todas las plataformas de la Tabla 2.2 son técnicamente viables para una arquitectura Kappa, puesto que ofrecen registro persistente, reproducción por desplazamiento y consumidores duraderos; requisitos fundamentales que exige todo diseño.

En la capa de procesamiento de flujos, Apache Flink se ha consolidado como la mejor opción para el cómputo en tiempo real usando la semántica de eventos, mientras que Kafka Streams ocupa un nicho más acotado como librería embebida para aplicaciones Java/Scala nativas de Apache Kafka que no requieren un clúster de procesamiento independiente. Apache Spark Structured Streaming, por su parte, se presenta como el estándar de la industria para el procesamiento por micro-lotes, apoyado en la amplitud de su ecosistema y la unificación del procesamiento en lotes y en streaming bajo una misma API; característica que comparte Apache Flink pero que en Spark se beneficia de una integración más amplia con el ecosistema de almacenamiento *lakehouse* y herramientas SQL (S. Lakshminpathy [11]).

Para el almacenamiento de series temporales de los datos IoT, sistemas especializados como InfluxDB [12], TimescaleDB [13] y QuestDB [14] compiten por ofrecer el mejor equilibrio entre velocidad de escritura, compresión de datos y capacidad de consulta analítica; sin embargo conviene advertir que buena parte de los benchmarks de rendimiento publicados provienen directamente de los creadores de cada producto como se aprecia en las citadas fuentes, por lo que dichos benchmarks deben interpretarse con cautela hasta que existan estudios independientes.

En la capa de visualización, Grafana se ha convertido en el estándar para paneles operacionales sobre datos de series temporales, en muchos casos combinado con Prometheus para monitoreo de infraestructura.

T. Domínguez-Bolaño et al. [15] complementan la panorámica anterior con un análisis comparativo de plataformas IoT de código abierto de propósito más amplio (como ThingsBoard,

openHAB, OpenRemote o SiteWhere) que integran varias de estas capas en un único producto, frente al enfoque modular de ensamblar componentes especializados por capa que caracteriza a los pipelines contruidos a medida. Ambos enfoques (plataforma integrada y ensamblaje modular de componentes) representan compromisos distintos entre rapidez de despliegue y flexibilidad de adaptación ante los requisitos específicos de un proyecto.

En conjunto, la evidencia revisada sugiere que las herramientas de código abierto en el ámbito de los datos IoT son hoy suficientemente maduras para competir con las ofertas propietarias en la nube en la mayoría de los casos de uso, a costa de una mayor carga operativa que exige experiencia técnica para configurar, escalar y asegurar cada componente del pipeline.

2.2.2. Retos abiertos: seguridad, interoperabilidad y especialización técnica

Pese a la madurez alcanzada por las herramientas disponibles, Y. Liu [1] y T. Domínguez-Bolaño et al. [15] plantean, de forma independiente, tres retos que permanecen abiertos:

- En primer lugar, la seguridad: la superficie de ataque se amplía con cada dispositivo IoT conectado, muchos de los cuales carecen de mecanismos de seguridad inherentes debido a la naturaleza distribuida de las redes IoT y a la ausencia de protocolos de seguridad estandarizados entre fabricantes [1].
- En segundo lugar, la interoperabilidad: la coexistencia de múltiples protocolos, formatos de datos y estándares entre fabricantes sigue dificultando la integración de dispositivos heterogéneos dentro de una misma plataforma [15].
- En tercer lugar, la especialización técnica requerida: la complejidad y la escala de los datos generados por los dispositivos IoT hacen necesario el uso de herramientas o plataformas especializadas para procesarlos de forma efectiva [1]. El manejo de estas herramientas puede exigir un nivel de dominio técnico que no siempre está disponible en equipos de tamaño reducido.

2.2.3. Criterios de selección de herramientas de procesamiento streaming

La elección de un sistema concreto para el procesamiento de flujos dentro de una arquitectura Kappa no tiene una respuesta universal: distintas herramientas optimizan frente a distintos compromisos, y la decisión debe fundamentarse en los criterios aplicables al proyecto de procesamiento de streaming, no en percepciones públicas o popularidad de una herramienta.

A continuación se plantean algunos criterios considerados y su aplicación al dominio del procesamiento de streaming en general:

- **Requisitos de latencia vs. cadencia de generación de datos:** si la frecuencia de emisión de los dispositivos es del orden de segundos o más lenta, un motor de procesamiento por micro-lotes no introduce una degradación perceptible de la información generada; ya que la ventaja de latencia sub-segundo con un motor de streaming nativo solo es relevante cuando el caso de uso exige control reactivo o detección de eventos críticos en tiempo real.

- **Madurez del ecosistema y de las APIs en el lenguaje de implementación:** la profundidad de soporte de un lenguaje concreto (Java, Scala, Python) varía entre motores; un ecosistema con API nativa más madura reduce fricción de desarrollo y mantenimiento.
- **Unificación del modelo de procesamiento batch/streaming:** algunos motores permiten expresar lógica de procesamiento por lotes y en streaming bajo una misma API, lo que simplifica la arquitectura cuando ambos modos de procesamiento coexisten en el sistema.
- **Disponibilidad de talento y curva de adopción:** la disponibilidad de profesionales con experiencia previa en una tecnología concreta condiciona la viabilidad de mantenimiento y escalado de un sistema en producción a largo plazo.

Específicamente, en el dominio del procesamiento de streaming de dispositivos IoT industriales, la cadencia habitual de generación de eventos (normalmente en la escala de segundos) sitúa las exigencias de latencia del sistema por debajo del umbral en el que las ventajas de un motor de streaming resultan necesarias. Por tanto, un motor de procesamiento por micro-lotes con ventanas de agregación temporal ofrece garantías de tolerancia a fallos y semántica *exactly-once* equivalentes a las de un motor de streaming nativo, sin incurrir en complejidades operativas adicionales. Este análisis no excluye la idoneidad de los motores de streaming nativos en escenarios de mantenimiento predictivo, sistemas de control reactivo o detección de eventos críticos en tiempo real, donde la latencia resulta un factor determinante del diseño.

2.3 Contexto y justificación

El estado del arte revisado en la sección anterior pone de manifiesto un patrón recurrente: la mayor parte de las referencias sobre arquitecturas Kappa para IoT disponibles hoy (tutoriales de ingeniería, publicaciones de blog técnico, documentación de proyectos open-source) tienen un carácter fundamentalmente demostrativo. Su objetivo es ilustrar un concepto (por ejemplo, monitoreo de temperatura con Kafka, Spark Streaming y Streamlit, o un pipeline equivalente con StarRocks y MinIO), más que ofrecer una implementación completa y reproducible. Esta brecha entre la abundancia de ejemplos ilustrativos y la escasez de implementaciones reproducibles constituye la primera motivación de este trabajo: construir, documentar y poner a disposición pública un pipeline IoT completo, containerizado y basado exclusivamente en herramientas de código abierto, que sirva tanto como trabajo académico evaluable, como referencia técnica reutilizable por terceros.

El proceso de revisión de la literatura evidenció vacíos de información que este proyecto puede contribuir a complementar, al menos parcialmente, desde la práctica: no se encontró un trabajo independiente que documente el comportamiento de un broker MQTT de perfil ligero (como Eclipse Mosquitto) acoplado en la capa de ingesta a un microservicio de *bridging* dedicado hacia Kafka; combinación de elementos que la bibliografía consultada menciona de forma tangencial y sin evaluación.

La gobernanza de esquemas mediante Avro y un registro como Apicurio está documentada como buena práctica en arquitecturas de streaming de propósito general (en particular M. Kleppmann [16] describe el fundamento conceptual del registro de esquemas en tiem-

po de ejecución), pero no aparece integrada como componente explícito en ninguna de las implementaciones Kappa-IoT de código abierto revisadas ([6], [7]) ni en los trabajos académicos consultados sobre analítica y arquitectura IoT ([1], [5], [15]). Incorporar esta gobernanza aporta un elemento diferenciador respecto a las implementaciones revisadas, permitiendo la evolución controlada del esquema de telemetría a medida que cambian los tipos de sensor o los campos capturados, y acercando este pipeline a las prácticas ya consolidadas en arquitecturas de streaming de propósito general.

El diseño de doble sumidero (dual sink: métricas agregadas por ventana temporal hacia TimescaleDB para consumo en Grafana, y eventos individuales hacia PostgreSQL para consumo analítico) responde a una necesidad práctica de separar el consumo operacional en tiempo casi real (monitorización de planta, alertas) del consumo analítico orientado a negocio (reporting, análisis histórico, cruces con otras fuentes corporativas). Este patrón de doble sumidero está documentado en la industria de servicios gestionados propietarios (por ejemplo, el caso de AWS documentado por D. Kim et al. [17]) pero no aparece implementado en ninguna de las arquitecturas Kappa de código abierto revisadas, las cuales suelen asumir un único sumidero de salida.

En conjunto, estos elementos (implementación completa y reproducible, gobernanza de esquemas mediante Avro/Avro, y separación de sumideros operacional/analítico) justifican la elección de la arquitectura desarrollada en este trabajo no como una simple aplicación de una solución ya estudiada, sino como una propuesta que extiende la arquitectura Kappa hacia una dirección que la literatura actual documenta de forma incompleta.

2.4 Planteamiento del problema

De la revisión anterior se desprende que, si bien existen múltiples implementaciones ilustrativas de arquitecturas Kappa aplicadas a IoT, y sistemas equivalentes de gobernanza de esquemas y doble sumidero documentados en contextos de streaming de propósito general, ninguna implementación Kappa-IoT open-source revisada combina simultáneamente (i) un broker MQTT ligero acoplado a un microservicio de bridging dedicado hacia Kafka; (ii) un registro de esquemas Avro que valide el formato de cada evento transmitido y su admisibilidad entre la capa de ingesta y la capa de procesamiento; y (iii) una estrategia de doble sumidero que separe el consumo de datos operacionales en tiempo real del consumo analítico.

La mayoría de los recursos formativos y de referencia disponibles para diseñar este tipo de pipelines dependen, en algún punto de la cadena, de servicios gestionados o plataformas propietarias (colas de mensajería en la nube, motores de streaming como servicio, o plataformas de ingesta IoT gestionadas como AWS IoT Core o Azure IoT Hub), lo que dificulta la adopción en escenarios que requieren control total sobre el despliegue, la no dependencia de un proveedor concreto, o la posibilidad de ejecutar el pipeline en un entorno local (o on-premise) mediante contenedores.

Considerando lo anterior, el problema planteado en este trabajo es *la falta de una implementación de referencia, open-source y containerizada, de una arquitectura Kappa para IoT*

que integre bridging MQTT-Kafka, gobernanza de esquemas mediante Avro/Avro, y una estrategia de doble sumidero que separe el consumo operacional del analítico.

En el desarrollo del trabajo se utilizará un conjunto de datos de telemetría de consumo energético como caso de estudio, evaluando la viabilidad técnica de la arquitectura y dejando como elementos reutilizables el código, la configuración de despliegue y la documentación asociada.

Capítulo 3. OBJETIVOS

3.1 Alcance del proyecto

Este trabajo abarca el diseño, desarrollo y validación de un sistema desacoplado de microservicios para la ingesta, procesamiento, análisis y visualización de datos de dispositivos IoT, utilizando como caso de uso el análisis de la telemetría de contadores de consumo energético en edificios con mediciones de electricidad, agua fría y agua caliente.

3.2 Tareas a desarrollar

Para cubrir el alcance descrito, se plantean las siguientes tareas:

- Analizar el estado del arte en las tecnologías, estrategias y retos actuales en el procesamiento y análisis de datos en streaming.
- Analizar las herramientas open-source disponibles para el procesamiento y análisis de datos de dispositivos IoT.
- Diseñar e implementar un simulador de dispositivos IoT para la transmisión de datos de telemetría.
- Diseñar e implementar una arquitectura basada en contenedores para la orquestación y ejecución de los siguientes microservicios:
 - Servicio para la ingesta de datos, mediante un gestor de mensajería que acepte protocolos de telemetría estándar.
 - Servicio de filtrado y transformación que alimente dos sumideros: una base de datos optimizada para series temporales con métricas ya agregadas por ventana, y una base relacional para eventos individuales con tablas de referencia.
 - Servicio de análisis que calcule índices de eficiencia energética, efectúe análisis operativo y detección de anomalías en datos de consumo.
- Diseñar un servicio de visualización mediante dashboards, para la visualización interactiva de los datos procesados.

3.3 Objetivos

Los objetivos concretos de este trabajo se definen a continuación, acompañados de un indicador clave de rendimiento (KPI) que permite verificar su cumplimiento, y de las acciones necesarias para alcanzarlo. Los valores numéricos de cada KPI se han fijado a partir de rangos razonables de la práctica de ingeniería y de la literatura revisada en el estado del arte; se validarán y, si procede, se recalibrarán a partir de las pruebas piloto realizadas sobre la implementación.

3.3.1. Objetivo 1: Garantizar la ingesta fiable de telemetría en tiempo real

KPI: latencia inferior a 2 segundos para la ingesta del 95 % de los mensajes MQTT publicados; calculada desde la publicación del mensaje hasta su persistencia final, con una tasa de pérdida inferior al 0.1 %.

Acciones: desplegar Mosquitto como broker MQTT y un microservicio de bridge MQTT-Kafka encargado de reenviar los eventos ingeridos al tópico de Kafka correspondiente; definir el nivel de calidad de servicio (QoS) y la topología de tópicos; instrumentar el simulador MQTT y el bridge para medir la latencia extremo a extremo; ejecutar pruebas de ingesta sostenida con el simulador para verificar el cumplimiento del KPI.

3.3.2. Objetivo 2: Garantizar la gobernanza y evolución controlada del esquema de datos

KPI: el 100 % de los eventos ingeridos se validan correctamente contra el esquema Avro registrado; 0 % de fallos de deserialización ante cambios de esquema compatibles.

Acciones: definir el esquema Avro inicial de telemetría; configurar las reglas de compatibilidad de esquema en Apicurio; diseñar y ejecutar al menos una prueba de evolución de esquema verificando que no se interrumpe el flujo ni se pierden mensajes; documentar los límites de la protección que ofrece el registro.

3.3.3. Objetivo 3: Procesar y enriquecer los datos en streaming con baja latencia

KPI: latencia de procesamiento por micro-lote en Spark Structured Streaming inferior a 3 segundos tras el cierre de cada ventana; throughput sostenido de al menos 50 eventos por segundo sin acumulación de retraso entre lotes sucesivos.

Acciones: implementar el job de Spark Structured Streaming (DataFrame API) con ventanas de agregación temporal y *watermarking* para el tratamiento de datos tardíos; instrumentar métricas de throughput y de duración de cada micro-lote (*batch duration*) del job; ejecutar pruebas de carga incremental hasta identificar el punto de saturación y verificar el cumplimiento del KPI dentro del rango definido.

3.3.4. Objetivo 4: Ofrecer visualización diferenciada operacional y analítica

KPI: dashboard de Grafana que muestre datos con una latencia inferior a 5 segundos; se dispone de al menos 1 dashboard en Power BI que cubra consumo energético, eficiencia operativa y detección de anomalías.

Acciones: diseñar el dashboard operacional de Grafana sobre TimescaleDB; conectar Power BI a los eventos enriquecidos de PostgreSQL; definir y calcular los índices de eficiencia energética requeridos por las tareas de análisis; validar la latencia de refresco de los dashboards.

3.3.5. Objetivo 5: Validar la resiliencia del sistema

KPI: ante el fallo de un servicio individual, el sistema se recupera en menos de 60 segundos sin pérdida de datos.

Acciones: simular la caída de un servicio (por ejemplo, el broker de mensajería o uno de los sumideros) y medir el tiempo de recuperación y la integridad de los datos gracias a la retención en Kafka; documentar los resultados obtenidos.

3.4 Beneficios del proyecto

El desarrollo de este sistema aporta, por un lado, un trabajo académico evaluable que documenta con rigor una implementación completa de una arquitectura Kappa para IoT basada en herramientas de código abierto, cerrando parcialmente los vacíos identificados en el estado del arte respecto a la gobernanza de esquemas y la separación de sumideros operacional y analítico.

Por otro lado, el pipeline resultante queda disponible como referencia replicable mediante contenedores y sin dependencia de proveedores propietarios, lo que facilita su adopción como punto de partida en proyectos similares de monitorización energética.

Capítulo 4. ÉTICA Y COMPROMISO SOCIAL

Este capítulo recoge el compromiso del trabajo con los principios de igualdad de género y su contribución a los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 de Naciones Unidas.

4.1 Igualdad de género

Este trabajo se alinea con los principios de igualdad de género y respeto a la diversidad. Por su naturaleza (el desarrollo de un pipeline de procesamiento de datos de telemetría energética a partir de un conjunto de datos público de mediciones de contadores, sin participación de sujetos humanos) no requiere consideraciones de muestreo poblacional ni plantea riesgos de sesgo respecto a colectivos protegidos. Su desarrollo y redacción se han realizado garantizando un entorno de trabajo respetuoso, con tolerancia cero hacia cualquier forma de discriminación, y empleando un lenguaje inclusivo y no sexista en toda la memoria.

El ámbito de la ingeniería de datos y de las infraestructuras de código abierto sigue siendo, en la actualidad, un entorno con una representación desigual de mujeres, tanto en su historia (con frecuencia invisibilizada, desde las pioneras de la programación hasta las primeras analistas de sistemas) como en su composición actual. Iniciativas como Women in Big Data o los programas de mentoría en comunidades de software libre buscan corregir ese desequilibrio estructural.

Este trabajo contribuye modestamente en esa dirección por la vía de la accesibilidad: al basarse exclusivamente en herramientas de código abierto y en un conjunto de datos público, y al documentar de forma completa y reproducible su metodología (arquitectura, decisiones de diseño y procedimientos de medición), elimina barreras económicas y de acceso privilegiado a la información que tradicionalmente han filtrado quién puede participar en el aprendizaje y la práctica de la ingeniería de datos en streaming. Un pipeline que cualquiera puede clonar, ejecutar, estudiar y extender sin depender de licencias comerciales ni de servicios gestionados de pago es, por construcción, más accesible que uno reservado a quienes ya cuentan con presupuesto o acceso privilegiado dentro de la industria.

4.2 Objetivos de Desarrollo Sostenible

Este trabajo se relaciona principalmente con los Objetivos de Desarrollo Sostenible 9 (Industria, Innovación e Infraestructura), 7 (Energía Asequible y No Contaminante) y 11 (Ciudades y Comunidades Sostenibles).

Respecto al ODS 9, el proyecto construye una infraestructura de procesamiento de datos en tiempo real íntegramente basada en tecnologías de código abierto (Mosquitto, Apache Kafka,

Apicurio, Apache Spark, TimescaleDB, PostgreSQL, Grafana, Docker), documentada y desplegable mediante contenedores, cuyo patrón arquitectónico (ingesta MQTT, gobernanza de esquemas y doble sumidero operacional/analítico) es directamente transferible a otros sectores industriales que requieran monitorización de dispositivos IoT, como el mantenimiento predictivo o la telemetría de flotas. La validación metodológicamente honesta de sus resultados (pruebas de carga y de recuperación ante fallos medidas y reproducibles, en lugar de cifras estimadas) promueve además una cultura de innovación rigurosa y transparente.

Respecto al ODS 7, el sistema desarrollado procesa telemetría de consumo eléctrico y de agua (fría y caliente) de edificios, calculando índices de eficiencia energética, perfiles de carga y detección de anomalías de consumo. Este tipo de instrumentación es la base técnica sobre la que se apoyan las estrategias de gestión activa de la demanda y de reducción del consumo fantasma, contribuyendo indirectamente a un uso más eficiente y sostenible de la energía en el parque de edificios.

Respecto al ODS 11, la monitorización granular del consumo energético de edificios (a nivel de emplazamiento, tipo de medidor y ventana temporal) constituye una capacidad habilitadora para la gestión sostenible de infraestructuras urbanas, permitiendo identificar patrones de consumo ineficiente y priorizar intervenciones de mejora energética en el parque construido.

De forma complementaria, el trabajo guarda relación con el ODS 4 (Educación de Calidad), en la medida en que el sistema completo (código, configuración de despliegue y documentación de las decisiones de diseño y de los resultados medidos) queda disponible como material formativo reutilizable para la enseñanza de la ingeniería de datos en streaming aplicada a un caso de uso real.

Capítulo 5. DESARROLLO DEL PROYECTO

5.1 Planificación del proyecto

El proyecto se desarrolló a lo largo de tres meses, entre junio y agosto de 2026, mediante fases consecutivas e incrementales. El esfuerzo total fue de 330 horas, a un ritmo aproximado de 28 horas semanales (112 horas mensuales). La Tabla 5.1 recoge el cronograma con las actividades realizadas y su esfuerzo.

Actividad	Periodo	Horas	Entregable
Estudio de los antecedentes y estado del arte	Jun. 2026	30	Revisión bibliográfica
Búsqueda y adaptación de la fuente de datos; establecimiento de objetivos	Jun. 2026	5	<code>prepare_ashrae.py</code> , objetivos y KPIs
Selección de herramientas y diseño de la arquitectura	Jun. 2026	5	Documento de diseño
Desarrollo técnico y pruebas iniciales de la sección de procesamiento	Jun.–Jul. 2026	80	Artefactos en <code>bridge/</code> , <code>common/</code> , <code>docker/</code> , <code>schemas/</code> , <code>simulator/</code> y <code>spark/</code>
Validación, pruebas y ajustes de la sección de procesamiento	Jul. 2026	100	Informe de resultados: pérdida de mensajes (0,0000 %), latencia de ingesta p50/p95 (0,766 / 1,277 s), soporte de carga (652 sensores concurrentes, sin pérdida de mensajes), tiempo de recuperación ante fallo (4,1 s TimescaleDB / 5,1 s PostgreSQL)
Creación, pruebas y ajustes de la sección de visualización	Jul.–Ago. 2026	30	Artefactos en <code>grafana/</code> y <code>powerbi/</code>
Creación y prueba de scripts auxiliares	Ago. 2026	5	Artefactos en <code>tools/</code> y <code>setup.sh</code>
Redacción de la memoria	Jun.–Ago. 2026	75	Este documento
Total		330	

Tabla 5.1. Cronograma y esfuerzo de las actividades del proyecto

5.2 Descripción de la solución, metodologías y herramientas empleadas

5.2.1. Visión general de la arquitectura

El sistema implementa una arquitectura Kappa de flujo único con una bifurcación final hacia dos sumideros, tal y como muestra la Figura 5.1. Los componentes son servicios independientes, desacoplados mediante contenedores Docker, y la comunicación entre productores

y consumidores de datos se ejecuta usando Kafka como log reproducible, mientras que la gobernanza de esquemas se resuelve a través del registro Apicurio.

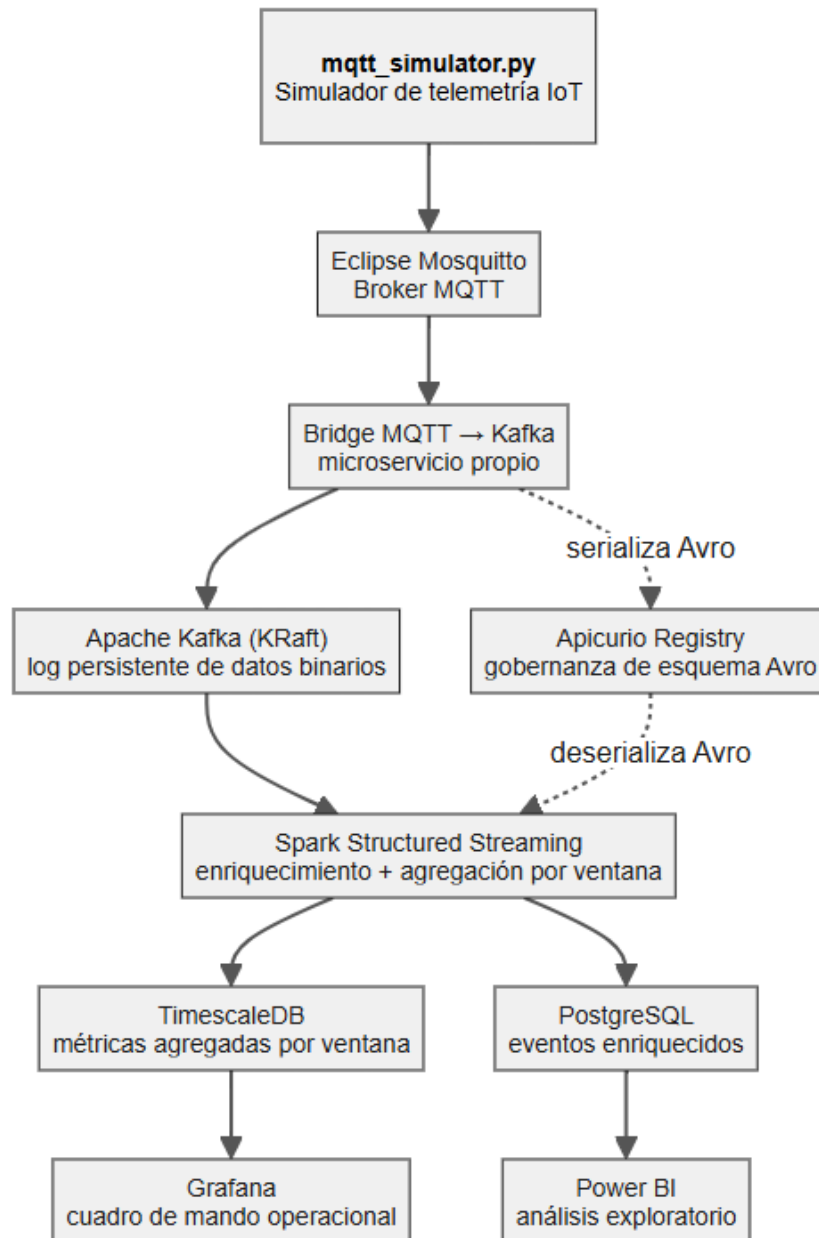


Figura 5.1. Arquitectura del sistema: flujo único Kappa (simulador, Mosquitto, bridge, Kafka, Spark) con bifurcación hacia el doble sumidero operacional y analítico. Elaboración propia.

El simulador (`mqtt_simulator.py`) publica telemetría hacia Eclipse Mosquitto, que actúa únicamente como broker de ingesta MQTT. El bridge, un microservicio de diseño propio, se suscribe a los mensajes desde Mosquitto, los serializa contra Apicurio Registry conforme al

esquema Avro vigente y los publica en Kafka.

La naturaleza de Kafka es de un log persistente de bytes crudos, ajeno a la estructura interna de los mensajes almacenados y distribuidos. Es por ello que el bridge, en la escritura, y Spark Structured Streaming, en la lectura, son quienes dialogan directamente con Apicurio para serializar y deserializar las cadenas de bytes almacenadas y distribuidas por Kafka.

Spark Structured Streaming consume cada mensaje mediante dos consultas independientes sobre el mismo flujo de Kafka; en una de ellas enriquece los mensajes y agrega métricas por ventana temporal hacia TimescaleDB para el consumo operacional en Grafana, y en la otra consulta guarda los mensajes hacia PostgreSQL para el consumo analítico con Power BI. Al tratarse de consultas independientes, cada una mantiene su propio *checkpoint* y su propia tolerancia a fallos, de modo que la caída de una consulta no interrumpe a la otra (sección ??).

5.2.2. Fuente de datos y variables utilizadas

Los datos para este proyecto provienen de la competición Kaggle ASHRAE – Great Energy Predictor III [18], un subconjunto del proyecto Building Data Genome 2 que registra el consumo energético por hora de 1.449 edificios reales en 16 emplazamientos desde el 2016-01-01 00:00h hasta el 2016-12-31 23:00h, con errores de medición y problemas de calidad (no hay datos sintéticos).

El script de preparación (`prepare_ashrae.py`) reduce el dataset original a un subconjunto de tres emplazamientos (identificados como 2, 3 y 5 con un total de 652 medidores y 5.682.185 lecturas de consumo), produciendo tres tablas Parquet: una tabla de hechos con la lectura de cada medidor (Tabla 5.2), una tabla de dimensión con los atributos de cada edificio (Tabla 5.3), y una tabla con el cálculo de los cuartiles según el histórico completo de medición de cada dispositivo (Tabla 5.4).

Variable	Uso en el sistema
<code>building_id</code>	Identificador del edificio; primer componente de la clave de partición en Kafka
<code>meter_type</code>	Tipo de medidor (electricidad, agua fría, agua caliente); segundo componente de la clave de partición en Kafka
<code>timestamp</code>	Instante en que se leyó el consumo; usado para el agregado por ventanas y el cálculo de ‘cierre de la ventana’ ante mensajes tardíos
<code>meter_reading</code>	Medición del consumo en kWh; usado para la detección de calidad, anomalías y métricas agregadas en los análisis

Tabla 5.2. Variables de la tabla de hechos (`ashrae_telemetry.parquet`) empleadas.

Variable	Uso en el sistema
building_id	Clave de unión con la tabla de hechos
site_id	Emplazamiento; primera clave de agrupación en las métricas agregadas
primary_use	Uso principal del edificio; segunda clave de agrupación de las métricas agregadas
square_feet	Superficie construida; base del cálculo de intensidad energética
year_built	Año de construcción; dejado para análisis futuros
floor_count	Número de plantas; dejado para análisis futuros

Tabla 5.3. Variables de la tabla de dimensión (`ashrae_buildings.parquet`) empleadas.

Variable	Uso en el sistema
building_id	Primera clave de unión con la tabla de hechos
meter_type	Segunda clave de unión con la tabla de hechos
baseline_p75	Percentil 75 histórico del medidor, usado para la detección de picos atípicos
baseline_iqr	Rango intercuartílico histórico del medidor

Tabla 5.4. Variables de la línea base por sensor (`ashrae_sensor_baseline.parquet`) empleadas.

5.2.3. Criterios de selección de herramientas

A continuación se documentan tres casos en los que se evaluaron herramientas usando los criterios planteados en el capítulo de Antecedentes y estado del arte, resultando en decisiones enmarcadas en los requisitos particulares de este proyecto.

Broker MQTT: Mosquitto con bridge propio frente a EMQX/HiveMQ Enterprise. Como recoge la Tabla 2.1, ningún broker MQTT con licencia aprobada por la Open Source Initiative ofrece un puente nativo hacia Kafka: esa función está disponible en las ediciones comerciales de EMQX [8] y HiveMQ [9]. Quedaban dos opciones compatibles con la filosofía de código abierto de este trabajo: adoptar un conector genérico como el conector MQTT de Stream Reactor para Kafka Connect [10], o desarrollar un microservicio de *bridging* propio. Se optó por la segunda opción porque permite asignarle al bridge responsabilidades adicionales más allá del simple reenvío de mensajes, sin depender de las limitaciones de configuración de un conector genérico. Adicionalmente, Eclipse Mosquitto se eligió sobre el resto de brokers de la Tabla 2.1 por ser la implementación de referencia de la Eclipse Foundation, con una huella de despliegue mínima adecuada para el volumen de mensajes de este proyecto.

Almacenamiento de series temporales: TimescaleDB frente a InfluxDB. La sección 2.2.1 advierte que los *benchmarks* de rendimiento entre los almacenamientos de series temporales proceden mayoritariamente de los propios fabricantes, por lo que la decisión en este proyecto no se apoyó en esas cifras sino en el ajuste con el resto de la arquitectura. Al ser TimescaleDB una extensión de PostgreSQL que ya se emplea en el sumidero analítico, ambos comparten cliente, dialecto SQL y patrón de escritura *upsert* en el mismo proceso de Spark, evitando

incorporar un lenguaje de consulta adicional (como Flux o InfluxQL) al proyecto.

Motor de procesamiento de flujos: Spark Structured Streaming frente a PyFlink. Aplicando los criterios de la sección 2.2.3 (relación entre la cadencia de los datos y los requisitos de latencia, madurez de la API en el lenguaje de implementación, unificación del modelo *batch/streaming*, y disponibilidad de talento) al caso concreto de este proyecto: el parque de medidores emite, verificado sobre el propio conjunto de datos, una lectura por hora y sensor, muy por debajo del umbral en el que las ventajas de latencia sub-segundo de un motor de streaming nativo como Flink resultan necesarias. A esto se suma que la API de PySpark tiene mayor madurez que la de PyFlink dentro del ecosistema Python, que Spark unifica el procesamiento por lotes y en streaming bajo el mismo modelo de *DataFrame*, y que su adopción en el ámbito profesional es más amplia.

5.2.4. Estructura del repositorio

```
tfm_boris_paternina/  
|-- setup.sh                # Crea el entorno virtual y prepara el dataset  
|-- requirements.txt        # Dependencias Python del pipeline completo  
|-- .sdkmanrc              # Versión de Java fijada (21 Temurin) para PySpark  
|-- pipeline/  
|   |-- data/  
|   |   |-- prepare_ashrae.py    # Preparación del subconjunto (ver Sección 5.2.2)  
|   |   |-- docker-compose.yml  # Orquesta Mosquitto, Bridge, Apicurio, Kafka,  
|   |   |                       # TimescaleDB, PostgreSQL y Grafana  
|   |   |-- docker/  
|   |   |   |-- mosquitto/      # Configuración del broker MQTT  
|   |   |   |-- grafana/        # Datasources y dashboards provisionados  
|   |   |   |-- timescaledb/    # Esquema SQL de métricas y progreso de streaming  
|   |   |   |-- postgres/      # Esquema SQL de eventos enriquecidos  
|   |   |-- simulator/  
|   |   |   |-- mqtt_simulator.py # Simulador del parque de medidores  
|   |   |   |-- simulator_helper.py # Carga y reparto del dataset para el simulador  
|   |   |-- bridge/  
|   |   |   |-- mqtt_kafka_bridge.py # Microservicio bridge MQTT -> Kafka (Objetivo 1)  
|   |   |   |-- Dockerfile       # Imagen del microservicio bridge  
|   |   |-- schemas/  
|   |   |   |-- telemetry_event_v1.avsc # Contrato de datos en Apicurio (Objetivo 2)  
|   |   |   |-- register_schema.py      # Registro y gobernanza de esquemas en Apicurio  
|   |   |-- spark/  
|   |   |   |-- stream_processing.py    # Spark Structured Streaming (Objetivos 3 y 4)  
|   |   |   |-- database_writers.py     # Escritura idempotente hacia los dos sumideros  
|   |   |   |-- monitoring.py           # Supervisión del proceso Spark  
|   |   |-- common/                    # Utilidades compartidas: logging, conexiones, Apicurio  
|   |   |-- logs/                      # Logs de ejecución  
|   |   |-- tools/                     # Scripts de medición (KPIs), prueba de fallos, demo  
|-- powerbi/  
|   |-- powerbi_dashboard_analitico.pbip          # Proyecto Power BI (formato PBIP)  
|   |-- powerbi_dashboard_analitico.Report/       # Definición de páginas y visuales  
|   |-- powerbi_dashboard_analitico.SemanticModel/ # Modelo semántico: tablas y relaciones
```

5.2.5. Preparación de los datos

Para la preparación de los datos usados en este proyecto, el script `prepare_ashrae.py` ejecuta, sobre los ficheros públicos de Kaggle, los siguientes pasos:

- **Filtrado por emplazamiento:** conserva únicamente los edificios cuyo `site_id` pertenece al subconjunto elegido (2, 3 y 5), descartando el resto mediante una unión provisional con la tabla de metadatos que actúa a la vez de filtro.
- **Decodificación del tipo de medidor:** traduce el código numérico original (0, 1 y 3) a su nombre (`electricity`, `chilledwater`, `hotwater`) usando la correspondencia declarada en la documentación de la competición.
- **Separación en tres tablas:** divide el resultado en la tabla de hechos, la tabla de dimensión y hace el cálculo de la tabla de cuartiles por sensor descritas en la sección 5.2.2 (Tablas 5.2, 5.3 y 5.4), cada una almacenada en un fichero Parquet independiente.
- **Verificación de la clave natural:** comprueba, en cada ejecución, que (`building_id`, `meter_type`, `timestamp`) identifica de forma única cada fila de la tabla de hechos; esta unicidad es la que garantiza la idempotencia del reprocesamiento de la arquitectura Kappa.
- **Reubicación temporal:** desplaza todos los timestamps del dataset original usando un único offset (`--fecha-final AAAA-MM-DD`) en el script Python, para que la última lectura caiga en la fecha indicada simplificando el análisis de los datos. Al ser un offset constante se conserva la cadencia y los ciclos diarios y estacionales del dataset original, y se exige una fecha presente o anterior para que ningún evento quede situado en el futuro.

5.2.6. Simulador de telemetría

El script `mqtt_simulator.py` ocupa el lugar de los 652 medidores reales representados en el subconjunto del dataset. Cada medidor publica una vez por hora, lo que supone 0,18 eventos por segundo considerando el total de medidores. Debido a lo anterior, el script usa el argumento `--acelerar` para incrementar el ritmo de publicación de los datos durante las pruebas de carga (en eventos por segundo), tal y como recoge la Tabla 5.5.

<code>--acelerar</code>	Cálculo eventos/segundo
2000	362
8000	1.448
40000	7.244

Tabla 5.5. Relación entre el factor de aceleración y el ritmo de publicación.

Una decisión de diseño relevante es publicar cada evento con un formato JSON en texto plano. El protocolo MQTT lo permite, ya que es agnóstico al formato del payload: no impone ninguna estructura ni codificación, y transporta indistintamente JSON, texto plano o binario.

```
{"building_id": "346", "meter_type": "electricity", "timestamp": "2026-02-13T17:00:00",  
"meter_reading": 40.99, "sim_publish_ts": 1788145201234}
```

La estructura de los campos de cada mensaje en formato JSON es la siguiente:

Campo	Significado
building_id	Identificador del edificio (texto)
meter_type	Tipo de medidor (electricity, chilledwater o hotwater)
timestamp	Timestamp al momento de tomar la lectura, sin zona horaria
meter_reading	Medición de consumo del medidor (decimal)
sim_publish_ts	Timestamp al momento de la publicación del dato, en formato epoch en milisegundos

Tabla 5.6. Campos del mensaje JSON publicado por el simulador.

En el terreno industrial/IIoT el formato más cercano al estándar es Sparkplug B, siendo entre un 50 y un 80 % más pequeño que el JSON equivalente al ser puramente binario. Por tanto se decide seguir la práctica recomendada: empezar con JSON en desarrollo, y migrar a un formato binario en producción cuando el ancho de banda lo exija.

5.2.7. Broker MQTT (Eclipse Mosquitto)

Mosquitto actúa únicamente como broker de ingesta MQTT: recibe la telemetría publicada por el simulador y la deja disponible para que el bridge la consuma, sin ejecutar ninguna lógica o transformación. Se despliega mediante la imagen oficial de Docker `eclipse-mosquitto:2.0.22`, con el puerto 1883 (MQTT, el que usa el simulador) y el 9001 (solo para inspección manual del tráfico durante el desarrollo del proyecto).

La configuración (`docker/mosquitto/mosquitto.conf`) fija cuatro aspectos relevantes:

Autenticación: `allow_anonymous true`. Válido en el entorno de desarrollo local del proyecto; en un despliegue real se sustituiría por `password_file` o autenticación TLS mutua.

Persistencia: `persistence true`, con almacenamiento y autoguardado cada 30 segundos. En MQTT, un mensaje publicado con QoS 1 ('al menos una vez') permanece pendiente en el broker hasta que el suscriptor lo confirma; la persistencia en disco es lo que permite que las sesiones y los mensajes QoS 1 todavía sin confirmar sobrevivan a un reinicio de Mosquitto, en vez de perderse junto con la memoria del proceso.

Límites de cola: `max_inflight_messages 200` fija cuántos mensajes QoS 1 puede retener el broker sin confirmar hacia un mismo suscriptor antes de detener el envío de más mensajes; esto es relevante para el bridge, que recibe la telemetría y debe confirmarla al mismo ritmo que la reenvía hacia Kafka. Por su parte, `max_queued_messages 10000` fija el tamaño de la cola de salida que el broker mantiene para cada suscriptor; es donde se acumulan los mensajes que el broker no ha podido entregar porque el suscriptor está momentáneamente desconectado, o porque estando conectado confirma los mensajes más despacio de lo que le llegan mensajes nuevos. Superado el límite de 10.000 mensajes en cola, los siguientes que lleguen para ese suscriptor se descartan.

Tamaño del mensaje: `message_size_limit 0` fija el tamaño máximo del payload que el broker acepta desde el publicador. Con 0 (el valor por defecto) no se impone ningún límite más allá del tope que ya fija el protocolo MQTT. Dado que el payload JSON del simulador `mqtt_simulator.py` ronda los 145 bytes en este proyecto, este límite solo se deja por claridad de la configuración.

El archivo `docker-compose.yml` comprueba el estado del contenedor con `mosquitto_sub` en lugar de solo verificar que el puerto responde, confirmando que el broker acepta conexiones antes de considerarlo saludable.

5.2.8. Bridge MQTT → Kafka

El bridge (`mqtt_kafka_bridge.py`) se suscribe a los mensajes `iot/#` en Mosquitto, valida cada mensaje contra el esquema Avro vigente en Apicurio Registry, lo serializa a una cadena de bytes y lo publica en Kafka. Los mensajes que no superen la validación no se descartan: se desvían al tópico de mensajes inválidos de Kafka (`iot.telemetry.dlq`) junto con el motivo del rechazo, el tópico MQTT de origen (por ejemplo `iot/346/electricity/telemetry`), y un timestamp de cuando fue procesado por el bridge.

Un mensaje es enviado a `iot.telemetry.dlq` por dos vías distintas: si no encaja en el esquema Avro (por un campo ausente, tipo de dato incorrecto, o un tipo de medidor que no esté entre los declarados en el esquema), o si no supera una validación adicional implementada en el propio bridge (como una fecha futura que dañaría el watermark de Spark, o una lectura negativa o no finita). El mensaje que se guarde en el tópico de inválidos de Kafka estará en su formato original para su posterior análisis o reprocesamiento.

Del lado de Kafka, el bridge se configura con `acks='all'`, el nivel de confirmación más estricto disponible. Cada vez que el bridge publica un mensaje, Kafka le devuelve una confirmación de escritura antes de que el bridge lo dé por entregado; el valor de `acks` decide qué tiene que ocurrir en Kafka antes de que esa confirmación salga hacia el bridge.

- Con `acks=0`, Kafka no envía ninguna confirmación: el bridge da el mensaje por entregado en el momento de enviarlo, sin manera de saber si realmente llegó.
- Con `acks=1`, Kafka confirma en cuanto el líder de la partición escribe el mensaje en su propio registro, antes de que las réplicas lo copien.
- Con `acks='all'`, Kafka espera a que todas las réplicas en sincronía también tengan el mensaje, y solo entonces envía la confirmación al bridge.

De esta forma se garantiza que los mensajes publicados por el bridge no se dan por confirmados hasta que ya están almacenados en todas las réplicas, evitando una potencial pérdida de mensajes ante una caída de un broker de Kafka.

Ante el posible escenario de una caída del microservicio de bridge, la sesión persistente de Mosquitto con `QoS 1` y `max_queued_messages 10000` retiene los mensajes hasta la reconexión del microservicio para continuar con la validación, serialización y publicación en Kafka.

La clave con la que el bridge publica cada mensaje en Kafka es el propio sensor (`building_id`:

meter_type), tal como se describió en la Tabla 5.2.

Para asegurar la futura escalabilidad del bridge, está disponible el argumento `--shared-group` que hace pedirle al broker Mosquitto que reparta los mensajes entre varias instancias del bridge en vez de duplicarlos para cada instancia por separado, por lo que este argumento permite escalar el bridge horizontalmente. Esto está pensado específicamente para su despliegue como contenedor en `docker-compose.yml`.

5.2.9. Registro de esquemas (Apicurio Registry)

Apicurio Registry gobierna el contrato de datos del proyecto. Se despliega mediante la imagen oficial de Docker `apicurio/apicurio-registry:3.3.1`, con una imagen aparte para la interfaz web (`apicurio/apicurio-registry-ui:3.3.1`).

Para que Apicurio pueda gobernar el esquema de validación de los datos, es necesario registrarlo antes del arranque del pipeline: se ejecuta manualmente la utilidad `register_schema.py`, que toma el contrato ya definido en `pipeline/schemas/telemetry_event_v1.avsc` y lo publica en Apicurio, estableciendo así, en un arranque desde cero, la primera versión del esquema en el registro.

`telemetry_event_v1.avsc` declara los cinco campos usados por el simulador de telemetría de la Tabla 5.6; por tanto la estructura de campos del esquema es la siguiente (se omitieron los valores de 'doc' en este ejemplo):

```
{
  "type": "record",
  "name": "TelemetryEvent",
  "namespace": "es.uea.tfm.iot",
  "doc": "...",
  "fields": [
    {
      "name": "building_id",
      "type": "string",
      "doc": "..."
    },
    {
      "name": "meter_type",
      "type": {
        "type": "enum",
        "name": "MeterType",
        "symbols": ["electricity", "chilledwater", "steam", "hotwater", "unknown"],
        "default": "unknown",
        "doc": "..."
      },
      "doc": "..."
    },
    {
      "name": "timestamp",
      "type": {"type": "long", "logicalType": "timestamp-millis"},
      "doc": "..."
    }
  ]
}
```

```
{
  "name": "meter_reading",
  "type": "double",
  "doc": "...",
},
{
  "name": "sim_publish_ts",
  "type": {"type": "long", "logicalType": "timestamp-millis"},
  "doc": "...",
}
]
}
```

El que se registre el esquema como un paso manual e independiente sigue una práctica recomendada en la industria para entornos de producción. Dejar que cada productor de mensajes registre automáticamente su propio esquema en Apicurio es cómodo en principio, pero delega el control de la evolución de esquemas en vez de ser un proceso supervisado. Por tanto, la práctica recomendada es publicar los esquemas mediante un mecanismo controlado y externo a la aplicación [19].

Los aspectos más importantes del funcionamiento de Apicurio dentro del pipeline son los siguientes:

- Apicurio comprueba que cada esquema que se le envía para su registro sea sintácticamente válido para serializar los mensajes a Avro. Adicionalmente comprueba que cada esquema nuevo sea compatible con todas las versiones de esquemas anteriores, no solo con el último. Si el esquema nuevo no pasa estas comprobaciones, se rechaza el registro.
- El bridge arranca y le pide el esquema más reciente a Apicurio: hace una llamada por HTTP a la API (`get_latest_version` sobre el subject `iot.telemetry.raw-value`) y recibe de vuelta dos cosas: el `schema_id` y el contenido del esquema. Esto ocurre una sola vez, al arrancar el proceso, antes de leer el primer mensaje de Mosquitto.
- Spark hace su propia llamada por HTTP a Apicurio, para obtener copia de todos los `schema_id` existentes. El bridge y Spark no se coordinan entre sí para estas llamadas; cada uno le pregunta a Apicurio por su cuenta.
- A partir de ahí, ninguno de los dos vuelve a llamar a Apicurio por cada mensaje. Tanto el bridge como Spark guardan los esquemas en memoria y los reutilizan durante toda la ejecución.
- El bridge usa su copia del esquema más reciente dado por Apicurio para serializar cada mensaje válido en Avro, y le antepone la cabecera de cinco bytes (un byte mágico más el `schema_id`) antes de publicarlo en Kafka como cadena de bytes.
- Spark usa su copia de todos los esquemas para deserializar cada mensaje (cadena de bytes) que lee de Kafka, comparando el `schema_id` que trae la cabecera de cada mensaje contra el de los esquemas disponibles; si no encuentra el esquema, detiene el proceso en vez de deserializar con errores la cadena de bytes.
- Apicurio, por su parte, no guarda esquemas en memoria ni en una base de datos externa. Todos los esquemas registrados son almacenados en un tópico interno de Kafka (almacenamiento `kafkaql`), así que estos siguen disponibles aunque se reinicie el

contenedor de Apicurio.

5.2.10. Apache Kafka

Kafka es el broker de mensajería sobre el que se apoya la arquitectura Kappa de flujo único descrita en la visión general (sección 5.2.1): su naturaleza es la de un log persistente de bytes crudos, ajeno a la estructura interna de los datos almacenados y distribuidos. Se despliega en modo KRaft con la imagen oficial de Docker `apache/kafka:4.3.1` como un solo nodo para este proyecto, con lo cual el mismo nodo combina los roles de *broker* y *controller*. Al tratarse de un solo nodo, el factor de replicación es necesariamente uno, por tanto no hay réplicas de las particiones a las que Kafka pueda recurrir si el nodo cae. Es la topología adecuada para el desarrollo y las pruebas de este proyecto, pero un despliegue real exigiría más de un broker para tener tolerancia real a fallos.

Cada mensaje recibido por Kafka desde el bridge es almacenado en su propia partición mediante la clave de partición compuesta de `building_id:meter_type`, que identifica de forma única a cada medidor, garantizando que todas sus lecturas se almacenen en la misma partición y se procesen en el orden en que se publicaron, tal como lo necesitan las ventanas de agregación de Spark.

Se utilizan dos tópicos para el enrutamiento de los mensajes: el tópico `iot.telemetry.raw` contiene los mensajes válidos publicados por el bridge, y el tópico `iot.telemetry.dlq` contiene los mensajes rechazados, descritos en la sección 5.2.8. Estos tópicos se crean de forma automática; tanto sus nombres como el número de particiones y el factor de replicación se configuran con los valores establecidos en el archivo `pipeline/.env`.

Tanto el tópico `iot.telemetry.raw` como `iot.telemetry.dlq` retienen los mensajes durante siete días. Esa ventana es la que sostiene la recuperación sin pérdida de datos: un consumidor de datos de Kafka que se detiene puede reiniciarse sin perder eventos dentro de ese margen de tiempo.

En la configuración de despliegue para Kafka en `docker-compose.yml` se establecen dos conexiones distintas con los otros servicios del sistema: `INTERNAL (kafka:9092)`, que es usada por la imagen del bridge, y `EXTERNAL (localhost:29092)`, que es usada por Spark. Esto es necesario porque Spark no se ejecuta en un contenedor, como se explicará en la sección 5.2.11.

5.2.11. Apache Spark Structured Streaming

Spark Structured Streaming (`stream_processing.py`) es el único consumidor de Kafka: dos consultas de streaming independientes al tópico `iot.telemetry.raw` deserializan cada mensaje Avro usando el esquema correspondiente al `schema_id` del mensaje, los enriquecen cruzándolos con las tablas de referencia del proyecto, y los guardan en sus respectivos sumideros. Como cada consulta de streaming se realiza con su propio *checkpoint* que registra

hasta qué punto se ha consumido el tópico, la caída de una consulta causada por una falla de PostgreSQL o TimescaleDB no interrumpe a la otra.

Las dos consultas comparten la misma lógica de enriquecimiento inicial de los datos: un join por difusión de Spark (*broadcast join*) con `ashrae_buildings.parquet` y con `ashrae_sensor_baseline.parquet`, ya que ambas tablas son lo bastante pequeñas para replicarse en la memoria de cada nodo sin necesidad de barajar (*shuffle*) los datos. Adicionalmente, la consulta hacia TimescaleDB agrega por ventanas temporales de una hora sobre el tiempo de evento (`timestamp`), agrupando siempre por tipo de medidor además de por emplazamiento y uso del edificio. Esta agregación por ventana de TimescaleDB exige una marca de agua (*watermark*) que le indique a Spark cuánto puede tardar un evento en llegar antes de dar una ventana por cerrada. La consulta hacia PostgreSQL almacena cada evento enriquecido de forma directa, por lo que no necesita ventanas de agregación con *watermark*.

La escritura en ambos sumideros se hace a través del script `database_writers.py`: este ejecuta un UPSERT (`INSERT ... ON CONFLICT DO UPDATE`) sobre la clave natural de cada tabla, en vez de un simple `INSERT`. De esta forma se hace idempotente el reprocesamiento del log de Kafka, condición necesaria en una arquitectura Kappa. Ante una caída breve de PostgreSQL o TimescaleDB, `database_writers.py` reintenta un número configurable de veces antes de dar la conexión por perdida.

Spark se ejecuta desde el entorno virtual del host y no como un servicio más del `docker-compose.yml` a diferencia del resto de componentes del pipeline. Una razón es que cada cambio en el código habría exigido reconstruir o volver a copiar la imagen del contenedor, mientras que ejecutándolo en local la depuración desde el editor es directa. Por otra parte, contenerizar el proceso de Spark exige requisitos de procesador y memoria que ya están comprometidos para los otros componentes del pipeline. Por tanto la contenerización queda como una tarea pendiente para una eventual puesta en producción.

5.3 Recursos requeridos

En este apartado debes enumerar los recursos que has utilizado para la ejecución del proyecto (recursos técnicos, dispositivos, material de laboratorio, asistencia de expertos, etc.). Si bien, en la sección donde se describe Metodología y Herramientas empleadas, se describen en detalle las mismas, en esta sección, únicamente debes enumerarlas.

5.4 Presupuesto

El Presupuesto supone la evaluación económica total del proyecto. No olvides que tu tiempo también vale dinero, no sólo hay que incluir el coste de los materiales empleados. A continuación, se muestra un ejemplo de resumen de presupuesto. Puedes añadir entradas a esta tabla si lo necesitas, conforme a la naturaleza de tu trabajo. Todos los recursos requeridos indicados en el apartado anterior deberían estar recogidos en la tabla de costes, aunque hayan tenido coste cero.

Tipo de coste	Valor	Comentarios
Horas de trabajo en el proyecto	[Número de horas]	Añade entradas para otras personas que hayan participado en tu proyecto para contabilizar sus horas aproximadas.
Equipo técnico utilizado	€	Si el equipo no lo has tenido que adquirir, indica su valor aproximado en el mercado si lo tuvieras que adquirir nuevo. Utiliza una entrada en la tabla por cada equipo (desktop, servidor, etc.)
Software utilizado	€	Si el Software no lo has tenido que adquirir, indica su valor aproximado en el mercado si lo tuvieras que adquirir. Indica coste 0€ si es Software que no requiere pagar precio por licencia. Ejemplos: IDE de desarrollo, herramienta de análisis de datos, programa de diseño gráfico, etc.
Estudios e informes	€	Si has tenido que adquirir algún informe de consultora o de revista de investigación, indica el coste.
Materiales empleados	€	P. ej.: material de laboratorio, sensores, etc.

Tabla 5.7. Costes del proyecto



Figura 5.2. Esta es una imagen de ejemplo.

5.5 Viabilidad

Este apartado es opcional, pero es interesante que incluyas un breve análisis de viabilidad económica del proyecto (relación coste / beneficio), y un análisis de sostenibilidad a futuro del resultado de tu proyecto.

5.6 Resultados del proyecto

Conforme a los objetivos específicos, describe los resultados finales obtenidos. Para proyectos con un enfoque de desarrollo de producto, debes incluir el resultado de tu plan de pruebas. Puedes añadir una descripción de cambios durante el proyecto respecto a los objetivos iniciales, y comentarios que quieras incluir de cómo has desarrollado las distintas actividades.

Capítulo 6. DISCUSIÓN

Esta sección es más habitual en trabajos de tipo científico.^o de "investigación", donde uno presenta brevemente los resultados principales y los discute, pero también puede utilizarse en otro tipo de trabajos.

Puedes incluir secciones específicas para discutir cuestiones como: limitaciones del estudio, limitaciones de la tecnología empleada, cambios respecto a objetivos planteados inicialmente.

Puedes incluir respuestas a preguntas del tipo: ¿la metodología inicialmente pensada ha sido útil?, ¿he tenido que adaptarme a cambios a lo largo del proyecto?, ¿qué cambios han sido y cómo he adaptado el proyecto para poder manejar esos cambios?, ¿qué impacto ha tenido el resultado de mi proyecto?

Capítulo 7. CONCLUSIONES

7.1 Conclusiones del trabajo

Breve descripción objetiva del resultado en relación al objetivo general de tu proyecto.

7.2 Conclusiones personales

Describe tus impresiones y experiencia personal durante el desarrollo del proyecto, o destacar la importancia que tiene el tema para ti, lo que has aprendido, o la trascendencia que ha tenido para ti o para otros este proyecto.

Capítulo 8. FUTURAS LÍNEAS DE TRABAJO

La implementación actual del pipeline se ha desplegado íntegramente mediante Docker Compose, lo cual resulta adecuado para el alcance de este TFM (un entorno de desarrollo y validación en una única máquina), pero presenta limitaciones evidentes de cara a un despliegue productivo real: no ofrece escalado horizontal automático de los componentes con mayor carga (particiones de Kafka, ejecutores de Spark Structured Streaming), ni mecanismos nativos de auto-recuperación ante fallos de nodo.

Una línea de evolución natural del sistema es la migración de la orquestación hacia una distribución ligera de Kubernetes, manteniendo el resto de la arquitectura (Mosquitto, Kafka en modo KRaft, Apicurio, Spark Structured Streaming y los sinks duales TimescaleDB/PostgreSQL) sin cambios sustanciales, ya que todos los componentes están containerizados y son, en principio, portables a manifiestos de Kubernetes o a Helm charts.

Entre las distribuciones evaluadas como candidatas destacan dos por su bajo consumo de recursos y su idoneidad para escenarios de IoT y edge computing:

- **K3s** (Rancher/SUSE): una distribución certificada por la CNCF (Cloud Native Computing Foundation), lo que garantiza compatibilidad completa con la API estándar de Kubernetes, que sustituye *etcd* por SQLite en despliegues de un solo nodo y reduce drásticamente el consumo de memoria frente a Kubernetes estándar, lo que la convierte en una de las opciones más utilizadas en estudios comparativos de orquestación de contenedores para entornos de recursos limitados.
- **MicroK8s** (Canonical): distribución empaquetada como snap, con fuerte integración en el ecosistema Ubuntu (el mismo utilizado en el entorno de desarrollo de este proyecto) y un sistema de complementos (*addons*) que simplifica la activación de componentes como almacenamiento persistente o ingress.

Como trabajo futuro, se propone:

1. Migrar el stack de Docker Compose a manifiestos de Kubernetes (o Helm charts), comenzando por un clúster de un solo nodo sobre K3s o MicroK8s en el mismo entorno WSL/Ubuntu ya utilizado para el desarrollo.
2. Evaluar el escalado horizontal de los ejecutores de Spark Structured Streaming y de las particiones de Kafka bajo carga variable, midiendo consumo de CPU, memoria y latencia, replicando la metodología de estudios recientes que han comparado el rendimiento y la eficiencia de recursos de varias distribuciones ligeras de Kubernetes en escenarios de edge computing bajo esas mismas métricas [20] y de trabajos que han evaluado herramientas de orquestación de contenedores específicamente en dichos entornos [21].
3. Explorar la incorporación de mecanismos de auto-recuperación y despliegue continuo (GitOps) sobre el clúster resultante, como extensión natural del repositorio actual del proyecto.

Bibliografía

- [1] Y. Liu, «Data Analytics in the Internet of Things Era: Tools, Approaches, Challenges, and Solutions,» *Journal of ICT Standardization*, vol. 13, n.º 4, págs. 427-448, 2025. DOI: 10.13052/jicts2245-800X.1344.
- [2] N. Marz, *How to Beat the CAP Theorem*, Thoughts from the Red Planet blog post, 2011. dirección: <https://nathanmarz.com/blog/how-to-beat-the-cap-theorem.html>.
- [3] N. Marz y J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. 20 Baldwin Road, PO Box 761, Shelter Island, NY 11964, USA: Manning Publications Co., 2015, pág. 328, ISBN: 9781617290343.
- [4] J. Kreps, *Questioning the Lambda Architecture*, O'Reilly Radar blog post, 2014. dirección: <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>.
- [5] H. Cao y M. Wachowicz, «An Edge-Fog-Cloud Architecture of Streaming Analytics for Internet of Things Applications,» *Sensors*, vol. 19, n.º 16, pág. 3594, 2019. DOI: 10.3390/s19163594.
- [6] S. Shinde, *Kappa Architecture Unveiled: Real-Time IoT Innovation and Hands-On Implementation*, Level Up Coding blog post, mar. de 2025. dirección: <https://levelup.gitconnected.com/kappa-architecture-unveiled-real-time-iot-innovation-and-hands-on-implementation-d8ff57f37167>.
- [7] G. Zefkilis, *Kappa Architecture in Action: From Sensors to Dashboards with Kafka, Spark Streaming, StarRocks, MinIO, and Docker*, Data Engineer Things blog post, 2025. dirección: <https://blog.dataengineerthings.org/kappa-architecture-in-action-from-sensors-to-dashboards-with-kafka-spark-streaming-starrocks-80492a509e1a>.
- [8] EMQ Technologies, *Data Integration*, Documentacion oficial de EMQX Enterprise. El puente de datos hacia Kafka no forma parte de la edicion de codigo abierto. Consultado en agosto de 2026, 2026. dirección: <https://docs.emqx.com/en/emqx/latest/data-integration/data-bridges.html>.
- [9] HiveMQ, *HiveMQ Enterprise Extension for Kafka*, Documentacion oficial de producto. La extension requiere licencia comercial (HiveMQ Professional o Enterprise); la Community Edition, bajo Apache 2.0, no la incluye. Consultado en agosto de 2026, 2026. dirección: <https://www.hivemq.com/products/extensions/hivemq-kafka-extension/>.
- [10] Lenses.io, *Stream Reactor: Kafka Connect connectors*, Repositorio del proyecto, licencia Apache 2.0. Incluye el conector fuente MQTT para Kafka Connect. Consultado en agosto de 2026, 2026. dirección: <https://github.com/lensesio/stream-reactor>.

- [11] S. Lakshmipathy, *Apache Flink vs Apache Kafka Streams vs Apache Spark Structured Streaming: Comparing Stream Processing Engines*, Onehouse blog post, abr. de 2025. dirección: <https://www.onehouse.ai/blog/apache-spark-structured-streaming-vs-apache-flink-vs-apache-kafka-streams-comparing-stream-processing-engines>.
- [12] InfluxData, *InfluxDB 3.0 vs OSS Benchmarks*, Documento oficial de benchmarking, 2025. dirección: <https://get.influxdata.com/rs/972-GDU-533/images/InfluxDB-3-0-vs-OSS-Benchmarks.pdf>.
- [13] TigerData, *TimescaleDB vs. InfluxDB*, Blog oficial de TigerData, creador de TimescaleDB, 2026. dirección: <https://www.tigerdata.com/blog/timescaledb-vs-influxdb-for-time-series-data-timescale-influx-sql-nosql-36489299877>.
- [14] QuestDB, *InfluxDB 3 Core Benchmarks: QuestDB Comparison*, Blog oficial de QuestDB, 2025. dirección: <https://questdb.com/blog/influxdb3-core-benchmarks/>.
- [15] T. Domínguez-Bolaño, O. Campos, V. Barral, C. J. Escudero y J. A. García-Naya, «An overview of IoT architectures, technologies, and existing open-source projects,» *Internet of Things*, 2022. DOI: 10.1016/j.iot.2022.100626.
- [16] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. 1005 Gravenstein Highway North, Sebastopol, CA 95472, USA: O'Reilly Media, 2017, cap. 4, ISBN: 9781449373320.
- [17] D. Kim, S. Park, Y. Park, V. Falconer e Y. Yun, *How SOCAR Built a Streaming Data Pipeline to Process IoT Data for Real-Time Analytics and Control*, AWS Big Data Blog, nov. de 2022. dirección: <https://aws.amazon.com/blogs/big-data/how-socar-built-a-streaming-data-pipeline-to-process-iot-data-for-real-time-analytics-and-control/>.
- [18] ASHRAE, *ASHRAE - Great Energy Predictor III*, Competición de Kaggle. Subconjunto del proyecto Building Data Genome 2 (BDG2), 2019. dirección: <https://www.kaggle.com/competitions/ashrae-energy-prediction>.
- [19] R. Yokota, *Best Practices for Confluent Schema Registry*, Blog oficial de Confluent, mar. de 2024. dirección: <https://www.confluent.io/blog/best-practices-for-confluent-schema-registry/>.
- [20] H. Koziolok y N. Eskandani, «Lightweight Kubernetes Distributions: A Performance Comparison of MicroK8s, K3s, K0s, and Microshift,» en *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE 2023)*, New York, NY, USA: ACM, 2023, págs. 17-29. DOI: 10.1145/3578244.3583737.
- [21] I. Čilić, P. Krivić, I. Podnar Žarko y M. Kušek, «Performance Evaluation of Container Orchestration Tools in Edge Computing Environments,» *Sensors*, vol. 23, n.º 8, pág. 4008, 2023. DOI: 10.3390/s23084008.

Capítulo 9. ANEXOS

Sirven para incluir documentación complementaria (planos, circuitos, código, ficheros de configuración, especificaciones técnicas y hojas de características, fichas explicativas, resultado de encuestas, reglamentación y normativas requeridas, etc.).